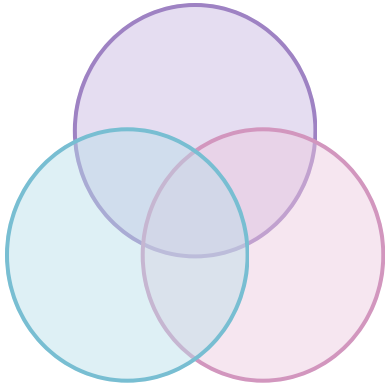


https://agencyai.mit.edu

agency *with* AI

Effective AI use in applied machine-learning projects

Lecture 6: Multi-Agent: The More the Merrier?

Shen Shen

Oct 27, 2026

2:30pm, Room 37-212

Slides in PDF

The more the merrier, or the messier?



Paul Degen · New Yorker Cover, 1985



Barry Blitt · New Yorker Cartoon, 2020

Sixteen Claude agents built a compiler, then got stuck

To stress test it, I tasked 16 agents with writing a Rust-based C compiler, from scratch, capable of compiling the Linux kernel. Over nearly 2,000 Claude Code sessions and $20,000 in API costs, the agent team produced a 100,000-line compiler that can build Linux 6.9 on x86, ARM, and RISC-V.

...

But when agents started to compile the Linux kernel, they got stuck. Unlike a test suite with hundreds of independent tests, compiling the Linux kernel is one giant task. Every agent would hit the same bug, fix that bug, and then overwrite each other's changes. Having 16 agents running didn't help because each was stuck solving the same task.

Same agents, same project.
When do more agents help, and when do they make a mess?

Building a C compiler with a team of parallel Claudes, Anthropic, Feb 2026

Outline

- A. Multi-agent in practice
- B. Dividing the work
- C. Coordinating the agents
- D. Running and checking the team

Claude's compiler project split the work by tests and by roles

Make parallelism easy

When there are many distinct failing tests, parallelization is trivial: each agent picks a different failing test to work on. After the test suite reached a 99% pass rate, each agent worked on getting a different small open-source project (e.g., SQlite, Redis, libjpeg, MQuickJS, Lua) to compile.

...

Multiple agent roles

Parallelism also enables specialization. LLM-written code frequently re-implements existing functionality, so I tasked one agent with coalescing any duplicate code it found. I put another in charge of improving the performance of the compiler itself, and a third I made responsible for outputting efficient compiled code. I asked another agent to critique the design of the project from the perspective of a Rust developer, and make structural changes to the project to improve the overall code quality, and another to work on documentation.

Building a C compiler with a team of parallel Claudes, Anthropic, Feb 2026

Claude Code Review splits review into finding and checking

Anthropic runs a team of review agents on nearly every internal pull request.

> When a PR is opened, Code Review dispatches a team of agents. The agents look for bugs in parallel, verify bugs to filter out false positives, and rank bugs by severity. The result lands on the PR as a single high-signal overview comment, plus in-line comments for specific bugs.
>
> …
>
> We've been running Code Review internally for months: on large PRs (over 1,000 lines changed), 84% get findings, averaging 7.5 issues. On small PRs under 50 lines, that drops to 31%, averaging 0.5 issues. Engineers largely agree with what it surfaces: less than 1% of findings are marked incorrect.

Devin Review: a reviewer with no shared context caught about 2 bugs per pull request, 58% severe.

Code Review for Claude Code, Anthropic, Mar 2026 · Also: Devin Review, Cognition, Apr 2026

Six weeks later: a bug got past human and agent review

Apr 2026, Anthropic's postmortem on a Claude Code bug:

> The changes it introduced made it past multiple human and automated code reviews, as well as unit tests, end-to-end tests, automated verification, and dogfooding.

Re-running Code Review on the faulty pull requests:

> As part of the investigation, we back-tested Code Review against the offending pull requests using Opus 4.7. When provided the code repositories necessary to gather complete context, Opus 4.7 found the bug, while Opus 4.6 didn't.

The agent reviewers missed it; a stronger model with the full codebase caught it.

An update on Claude Code quality reports, Anthropic, Apr 2026

Cursor's agents queued, dodged hard work, then drifted

First design: agents coordinated as equals through one shared file

1. Agents would hold locks for too long, or forget to release them entirely. Even when locking worked correctly, it became a bottleneck. Twenty agents would slow down to the effective throughput of two or three, with most time spent waiting.

Second design: no locks, and no hierarchy

We tried replacing locks with optimistic concurrency control. Agents could read state freely, but writes would fail if the state had changed since they last read it. This was simpler and more robust, but there were still deeper problems.

With no hierarchy, agents became risk-averse. They avoided difficult tasks and made small, safe changes instead. No agent took responsibility for hard problems or end-to-end implementation. This led to work churning for long periods of time without progress.

Later, from the engineer who led it

Some of these, like Skia for 2D graphics rendering or HarfBuzz for text shaping, were obvious choices. Others such as Taffy felt like they might go against the from-scratch goals of the project, since that library implements CSS flexbox and grid layout algorithms directly. This was not an intended outcome. 27:53

Scaling long-running autonomous coding, Cursor, Jan 2026 · FastRender interview, Simon Willison, Jan 2026

Cursor's fix: workers that don't talk to each other

Their third design, after a week of hundreds of agents building a browser from scratch.

- **Planners** continuously explore the codebase and create tasks. They can spawn sub-planners for specific areas, making planning itself parallel and recursive.
- **Workers** pick up tasks and focus entirely on completing them. They don't coordinate with other workers or worry about the big picture. They just grind on their assigned task until it's done, then push their changes.

At the end of each cycle, a judge agent determined whether to continue, then the next iteration would start fresh. This solved most of our coordination problems and let us scale to very large projects without any single agent getting tunnel vision.

Claude Research used the same lead-and-workers design.

Scaling long-running autonomous coding, Cursor, Jan 2026 · Also: Claude Research, Anthropic, Jun 2025

After: Cursor dropped its judge, fixed the brief, then added a coordinator

Feb 2026, Cursor's follow-up post:

> We found agents were reasonably good at following instructions to completion, so the judge was removed to keep the system simple.

The integrator every merge went through:

> It quickly became an obvious bottleneck. There were hundreds of workers and one gate (i.e. "red tape") that all work must pass through. We tried prompt changes, but ultimately decided it was unnecessarily and could be removed to simplify the system.

On the library the goal ruled out:

> Similarly, while the agents were told the project was a browser from scratch, they still pulled some dependencies they could've implemented themselves, or used as temporary scaffolding while proper implementation was underway. This was oversight in instructions. A later run explicitly laid out the dependency philosophy and which libraries must not be used, which corrected this.

Sep 2026, Cursor Projects:

> You oversee a Project by chatting with its coordinator agent. The coordinator doesn't write code itself but directs other agents that do.

Towards self-driving codebases, Cursor, Feb 2026 · Introducing Projects, Cursor, Sep 2026

Kimi's model learned to split the work itself

Trained with Parallel-Agent Reinforcement Learning (PARL), K2.5 learns to self-direct an **agent swarm** of up to **100 sub-agents**, executing parallel workflows across **up to 1,500 coordinated steps,** without predefined roles or hand-crafted workflows.

PARL uses a ***trainable orchestrator agent*** to decompose tasks into parallelizable subtasks, each executed by dynamically instantiated, ***frozen subagents***. Running these subtasks concurrently significantly reduces end-to-end latency compared to sequential agent execution.

• • •

Compared with a single-agent setup, this reduces execution time by up to **4.5x**.

By Kimi K2.6 (Apr 2026), the swarm scaled to 300 sub-agents and 4,000 steps.

Kimi K2.5 tech blog, Moonshot AI, Jan 2026 · Kimi K2.6 tech blog, Moonshot AI, Apr 2026

Kimi's orchestrator failed both ways: no split, or fake splits

The orchestrator's training reward, from the tech blog:

We define the reward as

$$r_{\text{PARL}}(x, y) = \lambda_1 \cdot \underbrace{r_{\text{parallel}}}_{\text{instantiation reward}} + \lambda_2 \cdot \underbrace{r_{\text{finish}}}_{\text{sub-agent finish rate}} + \underbrace{r_{\text{perf}}(x, y)}_{\text{task-level outcome}} .$$

The performance reward r_{perf} evaluates the overall success and quality of the solution y for a given task x. This is augmented by two auxiliary rewards, each addressing a distinct challenge in learning parallel orchestration. The reward r_{parallel} is introduced to mitigate *serial collapse*—a local optimum where the orchestrator defaults to single-agent execution. By incentivizing subagent instantiation, this term encourages the exploration of concurrent scheduling spaces. The r_{finish} reward focuses on the successful completion of assigned subtasks. It is used to prevent *spurious parallelism*, a reward-hacking behavior in which the orchestrator increases parallel metrics dramatically by spawning many subagents without meaningful task decomposition. By rewarding completed subtasks, r_{finish} enforces feasibility and guides the policy toward valid and effective decompositions. The hyperparameters λ_1 and λ_2 are annealed to zero over the course of training.

Kimi K2.5 tech blog, Moonshot AI, Jan 2026

A manager Devin assumed its child Devins knew what it knew

Cognition, maker of the Devin coding agent, on scaling Devin from single tasks to larger projects.

> This is live in Devin today. A manager Devin can break a larger task into pieces, spawn child Devins to work on them, and coordinate their progress through an internal MCP. Getting it to feel coherent took more context engineering than we expected. Managers trained on small-scoped delegation default to being overly prescriptive, which backfires when the manager lacks deep codebase context. Agents assume they share state with their children when they don't. Cross-agent communication, a sub-agent writing messages back to its manager to be passed to other agents in the agent team, doesn't happen by default, because models haven't been trained in environments where it needed to. Each of these took dedicated work to fix, and we're still improving on all of them.

Multi-agents: what's actually working, Cognition, Apr 2026

Five months later: Devin's lead writes each helper a brief

Fusion pairs a lead agent on a frontier model with a cheaper sidekick that writes code.

Jun 2026, where handing off failed:

Add a team selector to the search bar (cross-team search), gated on a flag.

`feature` `hard` `typescript`

- Hard, multi-file React/Redux feature graded on its judgment calls.
- Devin delegated the coding, and the subtle intent was lost.

When the judgment is the deliverable, delegating it backfires.

COST
-28% $6.84 → $4.91

SCORE
-27 54 → 27

Sep 2026, Fusion in Devin's desktop app:

The lead owns the plan, interpretation of ambiguity, and review. It hands the sidekick a brief of each task it delegates, with constraints and success criteria.

Instead of passing entire conversations between models, the lead and sidekick only exchange briefs, results, and feedback.

Devin Fusion, Cognition, Jun 2026 · Introducing Fusion in Devin Desktop & CLI, Cognition, Sep 2026

One agent's cleanup erased the others' work in a shared folder

A Claude Code user ran several agents in parallel in one working folder.

That single command wiped 17 files of other agents' work. The agent wanted to test its overlay.rs fix in isolation, so it reverted every other modified file back to HEAD. Irreversibly.

On top of that, agent-a92a75c was individually stashing other agents' files:

git stash -- crates/murk-propagator/src/pipeline.rs
git stash -- crates/murk-engine/src/overlay.rs
git stash -- crates/murk-engine/src/tick.rs

And agents a1a7075 and af08e8f both did git stash && cargo test && git stash pop — stashing ALL uncommitted changes globally just to run their own tests.

The four git reset HEAD reflog entries are the internal resets that git stash performs.

Root cause: parallel agents sharing one working directory used destructive git operations (checkout --, stash) to isolate their own changes for testing. These operations are global — they nuke every other agent's uncommitted work, not just the caller's.

Splitting the task does not isolate the agents' writes.

Claude has forgotten how to use git, Claude Code issue #26264, Feb 2026

Separate worktrees still leaked between subagents

Another Claude Code user gave about 14 subagents their own worktrees.

> The worktree-isolation "pin" (the state deciding which worktree a session's shell and git operations belong to) appears to be shared process-wide rather than per-agent. With several subagents active it reassigns constantly — sometimes between two consecutive tool calls in the same agent — so a command issued by agent A can execute inside agent B's worktree.

Isolation that depends on shared state isn't isolation.

One commit landed on main instead of a feature branch.

Aug 2026: Claude Code sessions can message each other; it's still up to each agent to avoid collisions.

Worktree isolation pin shared across parallel subagents, Claude Code issue #88950, Aug 2026 · Claude Code 2.1.224, Aug 2026

OpenAI had agent groups attack opposite versions of the problem

We used a system of coordinating agents powered by our internal model. The agents had access to tools such as the ability to read from a cached version of the internet and the ability to run code. Agents were subdivided into groups with the ability to communicate within the group. The groups varied in size, and the group that produced the Navier–Stokes resolution involved on the order of 10,000 concurrent agents. At all times we maintained the same strict safeguards that we apply to all our frontier model evaluations, including monitoring and isolation.

For each problem, we prompted different groups of agents with different variants of the problem statement, covering all variants of the problem. For the Navier–Stokes problem, we suggested versions "A" and "B" (particular forms of the Navier–Stokes problem which would result in a proof) and versions "C" and "D" (which would result in a disproof) to separate groups of agents.

On the Navier–Stokes Millennium Prize Problem, OpenAI, Sep 2026

OpenAI's own researcher gave multi-agent under 10% of the credit

Sep 2026, OpenAI researcher Noam Brown:

> There's one thing I want to make clear. The effort to solve a Millennium Prize Problem, this was not due to multi-agent. I wouldn't even attribute 10% of the credit to multi-agent. The reality is that OpenAI has trained a very powerful model. We can get that model to operate over very long horizons.

Brown on getting the agents to coordinate:

> In fact, it's actually very difficult to get these agents to coordinate in a productive way, because it's very tempting for them to just collapse to, "Oh, we're all just going to solve the problem independently." That is a local minimum that you can get stuck in.

Noam Brown – Agent swarms, alignment, & recursive self-improvement, Podcast, Sep 2026

An account from outside OpenAI: a team was steering the agents

I was shown a prompt and told the internal research model had simply been given the problem statement. Levent had been told by Sebastien "very little human input" had been used. This turned out not to be true. Over the course of the call, as members of their team sent Sebastien corrections and details over their internal chat, it emerged that an entire team had been working on the problem, that this was one of a number of things that was tried, that work had started on the unforced problem, that the team first set the model on easier problems, including Euler, that even the prompt that had been shown to me had been written by prompting Codex, and that an insane amount of compute had been used.

Statement on AI-assisted blowup proofs, Tristan Buckmaster, Sep 2026

Ramp's engineers run several attempts and keep one

Ramp, the finance company, built Inspect, an in-house background coding agent.

Because Inspect sessions are fast to start and effectively free to run, you can use them without rationing local checkouts or worktrees. A builder can kick off multiple versions of the same prompt, and just see which one lands. They can try different approaches or swap models without thinking twice. There's no limit to how many sessions you can have running concurrently, and your laptop doesn't need to be involved at all.

These are alternative answers to choose among, not parts we have to merge.

Jul 2026: Ramp added an orchestrator that splits one job across sandboxed agents.

Why we built our own background agent, Ramp, Jan 2026 · Zach Bruggeman, X, Jul 2026

A human set the agendas; a script ran the agents' meetings

A Stanford group gave an AI "principal investigator" a team of scientist agents and new COVID variants.

User

Immunologist, please provide your thoughts on the discussion (round 1 of 3). If you do not have anything new or relevant to add, you may say "pass". Remember that you can and should (politely) disagree with other team members if you have a different perspective.

Immunologist

As the Immunologist, it's my responsibility to ensure our design approach is rooted in a strong understanding of immunogenicity and potential cross-reactivity.

One team meeting, from the saved transcript. "User" is the script calling on the next agent.

Each team meeting ran 5 times in parallel; the principal-investigator agent then merged the five summaries into one answer.

Virtual Lab of AI agents designs SARS-CoV-2 nanobodies, Nature 2025 · Also: Co-Scientist, Google DeepMind, May 2026

Best Practices?

Share context? same vendor, a year apart	Cognition, Jun 2025: “share full agent traces.”	Cognition, Sep 2026: lead and sidekick “only exchange briefs, results, and feedback.”
Let workers talk?	Cursor, Jan 2026: workers “don’t coordinate with other workers.”	Claude Code agent teams, Feb 2026: teammates message each other.
A separate judge?	Anthropic’s app-building harness, Mar 2026: a separate evaluator agent.	Cursor, Feb 2026: dropped its judge.
Multi-agent at all? same vendor, 10 months apart	Cognition, Jun 2025: “Don’t build multi-agents.”	Cognition, Apr 2026: it works when “writes stay single-threaded.”

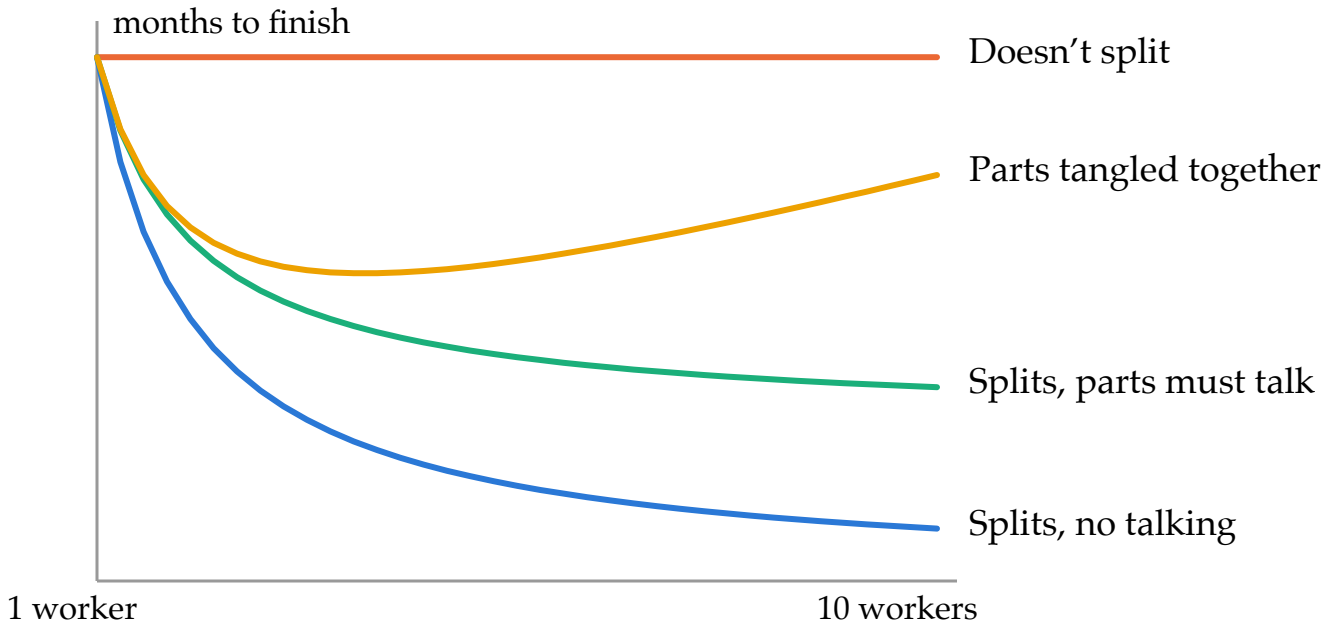
Each answer fit its task and its models at the time.

Outline

- A. Multi-agent in practice
- B. Dividing the work
- C. Coordinating the agents
- D. Running and checking the team

Whether more workers help depends on how the task divides

Fred Brooks, who ran IBM's OS/360, plotted months to finish against workers for four kinds of task.



- Keeps falling: double the workers, halve the time. Reaping wheat, each row on its own.
- Flat: the steps come in order, so extra workers wait. A baby takes nine months, however many are assigned.
- Falls, then levels off: briefing and agreeing cost time that never shrinks. One paper, four authors.
- Gets later: every part touches every other. Sixteen agents on one kernel bug.

> Men and months are interchangeable commodities only when a task can be partitioned among many workers *with no communication among them* (Fig. 2.1). This is true of reaping wheat or picking cotton; it is not even approximately true of systems programming.

The Mythical Man-Month, ch. 2, Addison-Wesley 1975

Divide the work by decisions, not by steps

David Parnas splits one small program two ways.

The usual way makes "each major step in the processing a module", as a flowchart would.

His way hides decisions:

On the Criteria To Be Used in Decomposing Systems into Modules

D.L. Parnas
Carnegie-Mellon University

Expected Benefits of Modular Programming

The benefits expected of modular programming are: (1) managerial—development time should be shortened because separate groups would work on each module with little need for communication:

We have tried to demonstrate by these examples that it is almost always incorrect to begin the decomposition of a system into modules on the basis of a flowchart. We propose instead that one begins with a list of difficult design decisions or design decisions which are likely to change. Each module is then designed to hide such a decision from the others.

For agents: give each agent one decision to own, and settle the interfaces between them before the work starts.

On the criteria to be used in decomposing systems into modules, CACM 1972

The builder changed the tests so the work split again

Back to the kernel bug. “I” is Nicholas Carlini, the Anthropic researcher running the agents.

> The fix was to use GCC as an online known-good compiler oracle to compare against. I wrote a new test harness that randomly compiled most of the kernel using GCC, and only the remaining files with Claude's C Compiler. If the kernel worked, then the problem wasn’t in Claude’s subset of the files. If it broke, then it could further refine by re-compiling some of these files with GCC. This let each agent work in parallel, fixing different bugs in different files, until Claude's compiler could eventually compile all files. (After this worked, it was still necessary to apply delta debugging techniques to find pairs of files that failed together but worked independently.)

Give each agent a different subset and each agent gets a different bug.
The harness made debugging more separable; interactions still needed joint tests.

The split came from the test harness, not from the agents. Making the work divisible was the person’s job.

Building a C compiler with a team of parallel Claudes, Anthropic, Feb 2026

Suppose we are tasked to work on a wildlife classifier

Task: 50,000 labeled camera-trap images from 40 sites.

Ship a species classifier and a report on how it does at sites it has never seen.

The work:

split	labels	train	score	report

For illustration, we adopt a common trap. Photos from one camera look almost the same, so split by site, not by image. However we divide the work, that rule has to be kept.

Brainstorm some possible ways to divide and conquer

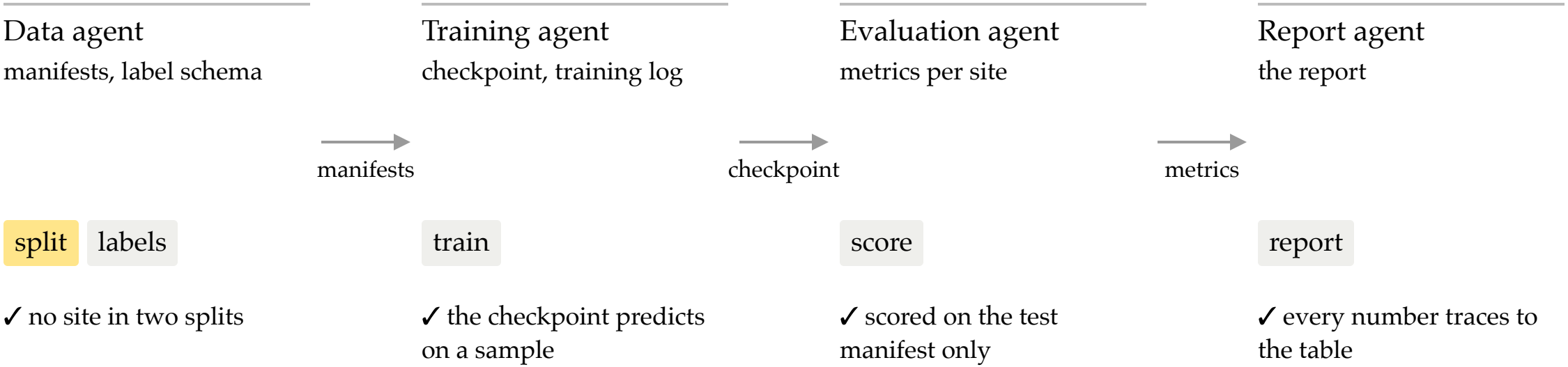
Pick the split, and we have picked the shape of what we deliver.

Conway, 1968:

> organizations "are constrained to produce designs which are copies of the communication structures of these organizations."

Split by stage: a pipeline of four agents

Each agent owns one step. What crosses between them is a file and its format.



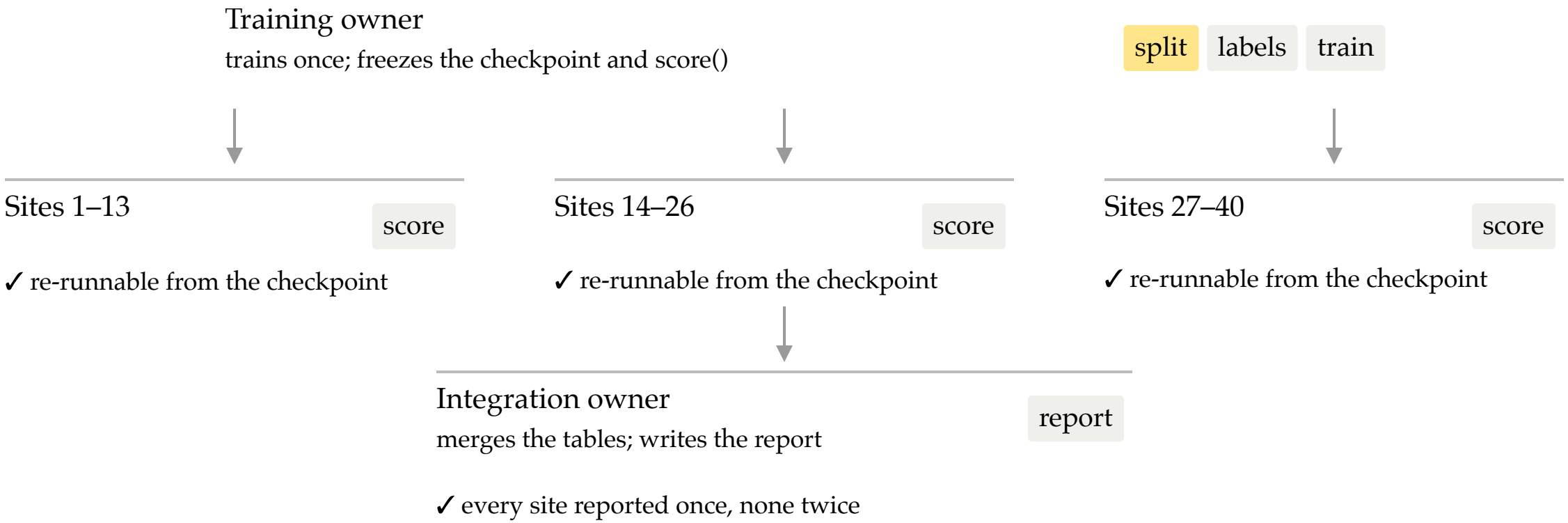
Fails when a split by image passes training and scoring; only the first check, or truly new sites, exposes it.

And the steps wait on each other: Brooks's Fig. 2.2, four agents, little speedup.

The Mythical Man-Month, ch. 2, Addison-Wesley 1975

Split by data: one frozen model, one agent per group of sites

Map and reduce. Training happens once; the checkpoint and score() go out, and one table per agent comes back.



Fails when a site agent “fixes” the model for its sites, and the shared model forks.

The compiler’s re-split, Feb 2026: each agent got its own files. Brooks’s Fig. 2.1: the part that splits cleanly speeds up.

Split by decision: each agent hides one choice

Information hiding, Parnas 1972. Each owner hides a decision likely to change, behind an interface agreed first.

Split owner	Label owner	Model owner	Evaluation owner
held-out sites; publishes the manifests	the label set; publishes the schema	architecture, recipe; publishes predict()	metrics, thresholds; publishes score()
split	labels	train	score report
✓ no site in two splits	✓ every label maps to the schema	✓ predict() runs on a sample	✓ score() runs on dummy predictions

Agreed before anyone starts: manifests · schema · predict() · score(). After that, nothing else crosses.

Fails when nobody owns a decision.

Cognition's illustrative Flappy Bird example (2025): agents chose incompatible visuals and behavior.

On the criteria to be used in decomposing systems into modules, CACM 1972 · Don't build multi-agents, Cognition, Jun 2025

Independent review: a second agent checks the work

Not a split. An arrangement you add to any of the three: the builder does not grade its own work.

Builder

any of the three splits

split | labels | train | score | report

The reviewer re-runs score() on the manifests.

artifacts, not a summary →

Reviewer, clean context

gets the task, the manifests, the code, the training log, the metrics, the repository

re-score

✓ a site in two splits?

✓ near-duplicate frames across splits?

✓ a test set touched in training?

✓ a pooled score hiding one site's collapse?

Fails when the reviewer gets a summary instead of the artifacts, or less context than the builder had.

Recall slide 7: a stronger model with the full codebase caught what the reviewers missed.

An update on Claude Code quality reports, Anthropic, Apr 2026 · The more you automate, the less you see, NeurIPS 2025

Choose a decomposition and a checking arrangement

	Runs in parallel?	What crosses between agents	What no single agent sees
By stage	No; the steps wait	A file and its format at each handoff	A leak made upstream
By data	Yes, the scoring	One checkpoint out, one table back	A problem that spans sites
By decision	Yes, once the interfaces are set	Only the interfaces	A decision nobody owns
+ Independent review	After the build, on any of the three	The artifacts, not a summary	The builder's own blind spots

For your project: which split? Who owns the train/test rule? What exactly does the reviewer get?

Why we built our own background agent, Ramp, Jan 2026

Outline

- A. Multi-agent in practice
- B. Dividing the work
- C. Coordinating the agents
- D. Running and checking the team

Coordination means managing dependencies between tasks

The Interdisciplinary Study of Coordination

THOMAS W. MALONE
KEVIN CROWSTON

A key insight of the framework presented here is that coordination can be seen as the process of managing dependencies among activities.

Table 1. Examples of Common Dependencies between Activities and Alternative Coordination Processes for Managing Them (Indentations in the left column indicate more specialized versions of general dependency types)

Dependency	Examples of coordination processes for managing dependency
Shared resources	"First come/first serve", priority order, budgets, managerial decision, market-like bidding
Task assignments	(same as for "Shared resources")
Producer / consumer relationships	
Prerequisite constraints	Notification, sequencing, tracking
Transfer	Inventory management (e.g., "Just In Time", "Economic Order Quantity")
Usability	Standardization, ask users, participatory design
Design for manufacturability	Concurrent engineering
Simultaneity constraints	[Scheduling, synchronization
Task / subtask	Goal selection, task decomposition

In our stories: the pipeline's steps on slide 30 are prerequisite constraints; the compiler's agents all hitting one kernel bug was a shared resource; Cursor's planners did task/subtask decomposition.

The interdisciplinary study of coordination, ACM Computing Surveys 1994

Why multi-agent runs failed: setup, handoffs, checks

Why Do Multi-Agent LLM Systems Fail?

Mert Cemri[1*] Melissa Z. Pan[1*] Shuyi Yang[2*] Lakshya A Agrawal[1] Bhavya Chopra[1]
Rishabh Tiwari[1] Kurt Keutzer[1] Aditya Parameswaran[1] Dan Klein[1]
Kannan Ramchandran[1] Matei Zaharia[1] Joseph E. Gonzalez[1] Ion Stoica[1]
[1]UC Berkeley [2]Intesa Sanpaolo *Equal Contribution

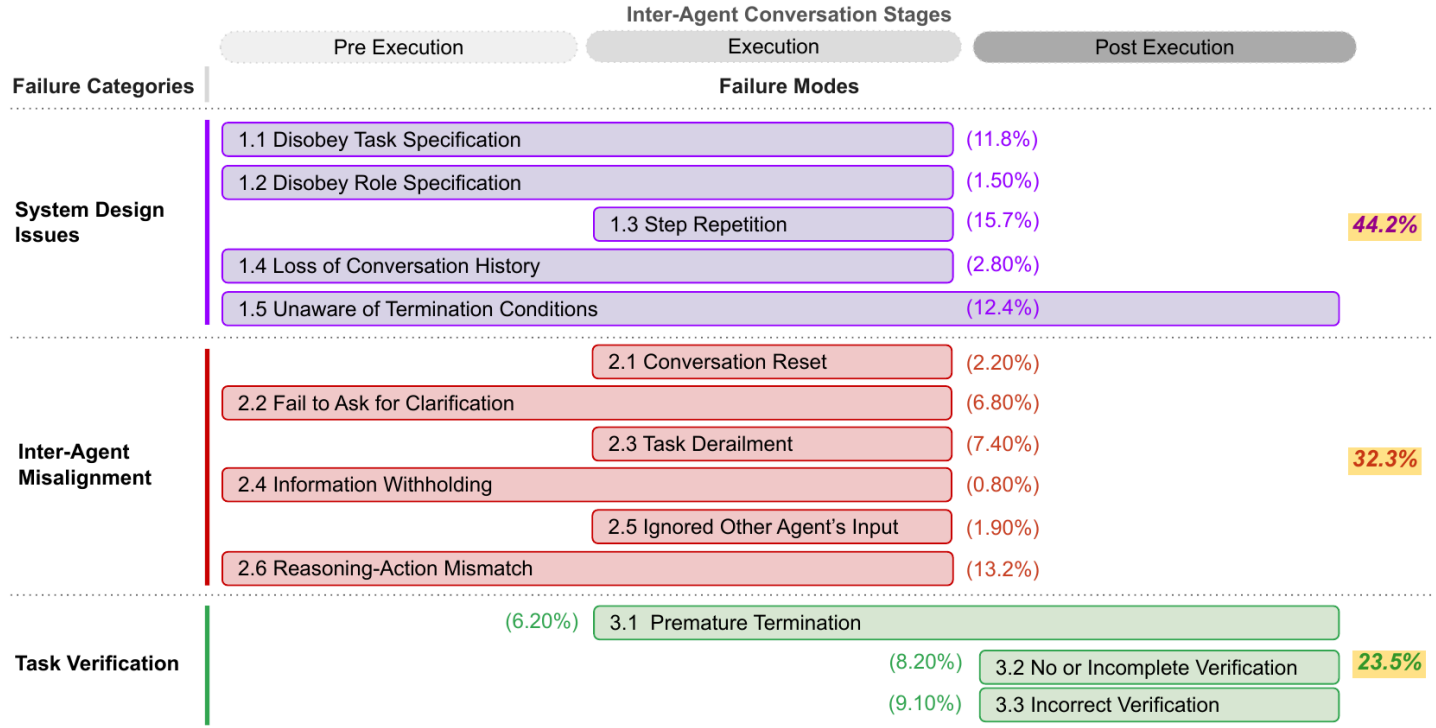


Figure 1: MAST: A **Taxonomy of MAS Failure Modes**. The inter-agent conversation stages indicate when a failure typically occurs within the end-to-end MAS execution pipeline. A failure mode spanning multiple stages signifies that the underlying issue can manifest or have implications across these different phases of operation. The percentages shown represent the prevalence of each failure mode and category as observed in our analysis of 1642 MAS execution traces.

Cognition's illustrative Flappy Bird example: context matters

> Suppose your Task is "build a Flappy Bird clone". This gets divided into Subtask 1 "build a moving game background with green pipes and hit boxes" and Subtask 2 "build a bird that you can move up and down".
>
> It turns out subagent 1 actually mistook your subtask and started building a background that looks like Super Mario Bros. Subagent 2 built you a bird, but it doesn't look like a game asset and it moves nothing like the one in Flappy Bird. Now the final agent is left with the undesirable task of combining these two miscommunications.

> *Principle 1*
>
> Share context, and share full agent traces, not just individual messages

...

> *Principle 2*
>
> Actions carry implicit decisions, and conflicting decisions carry bad results

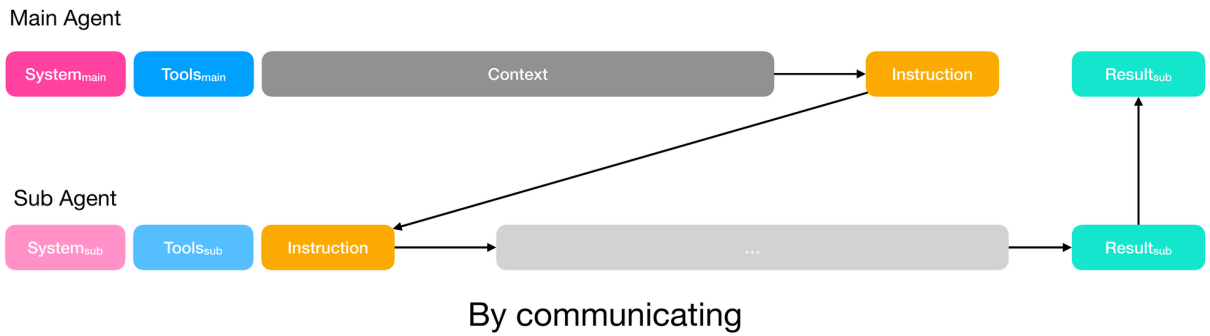
Don't build multi-agents, Cognition, Jun 2025

Manus hands a sub-agent an instruction, or forks the whole context

"Context"

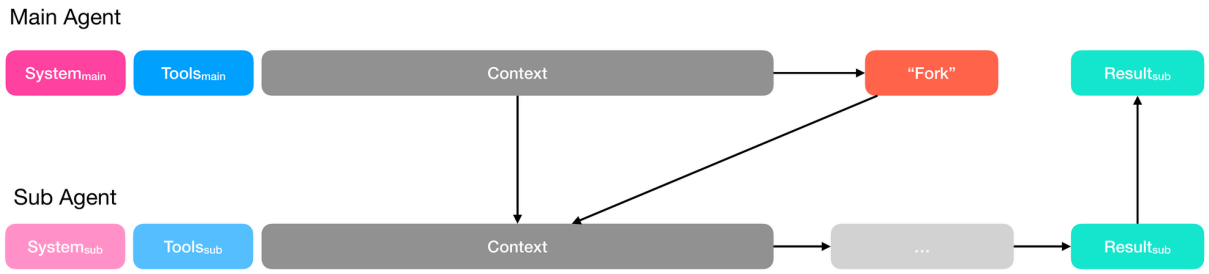
"~~Do not communicate~~ by sharing memory;
~~instead, share memory~~ by communicating."

Instruction only: its own prompt and tools, a result back. Cheap; it knows only the brief.



By communicating

Fork: inherits the parent's current context. More input context; token cost depends on caching.



By sharing context

Same model, six wirings: each task had a different winner

SCALING LARGE LANGUAGE MODEL-BASED MULTI-AGENT COLLABORATION

Chen Qian[⋆†], **Zihao Xie**[⋆†], **YiFei Wang**[⋆], **Wei Liu**[⋆], **Kunlun Zhu**[⋆], **Hanchen Xia**[⋆], **Yufan Dang**[⋆], **Zhuoyun Du**[⋆], **Weize Chen**[⋆], **Cheng Yang**[♣], **Zhiyuan Liu**[⋆], **Maosong Sun**[⋆✉]
⋆Tsinghua University ♣Peng Cheng Laboratory
qianc62@gmail.com xie-zh22@mails.tsinghua.edu.cn sms@tsinghua.edu.cn

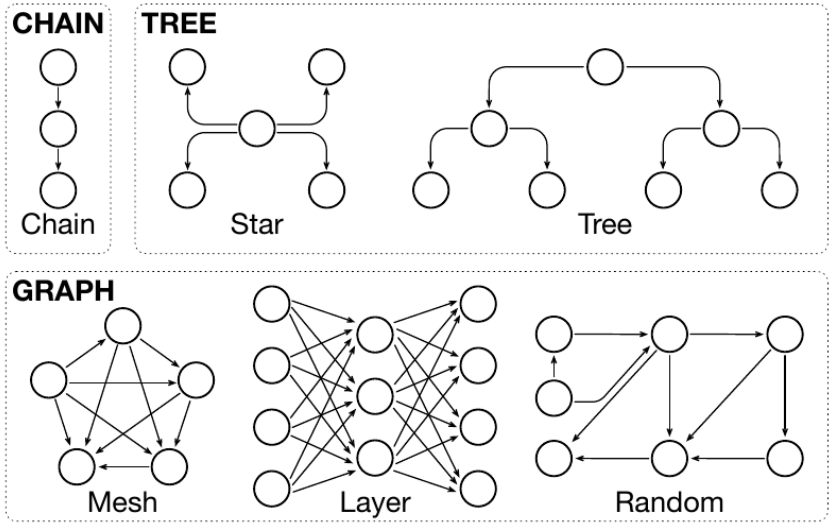


Figure 2: Representative topologies.

Each circle is an agent doing the work; each arrow is a reviewing agent that passes the work along.

Every row uses GPT-3.5. MMLU: a knowledge quiz. HumanEval: writing Python functions. SRDD: building whole programs. CommonGen: writing sentences from given words.

Method	MMLU	HumanEval	SRDD	CommonGen
MACNET-CHAIN	0.6632	0.3720	**0.8056**	0.5903
MACNET-STAR	0.4456†	0.5549†	0.7679†	0.7382†
MACNET-TREE	0.3421†	0.4878†	0.8044	**0.7718**†
MACNET-MESH	0.6825	0.5122†	0.7792†	0.5525†
MACNET-LAYER	0.2780†	0.4939†	0.7623†	0.7176†
MACNET-RANDOM	**0.6877**	0.5244†	0.8054	0.5912

Density Perspective Table 1 illustrates that different types of topologies vary significantly in effectiveness for specific tasks; no single topology consistently excels across all tasks. For instance, a chain topology is more suitable for software development, while a tree topology is ideal for creative

In these tests, merging versions reduced average quality

Same agents and tasks. Divergent: arrows fan out from the first agent.

Convergent: every arrow reversed, so several agents' versions meet at one agent, which merges them into one.

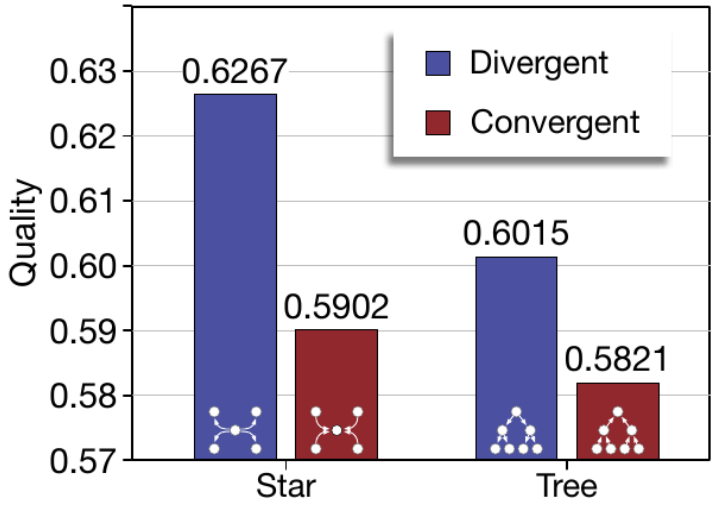


Figure 6: Comparison between topologies and their reversed counterparts.

Quality: the average over the four tasks. The axis starts at 0.57, so the gaps look larger than they are: about 4 points for Star, 2 for Tree.

Direction Perspective Beyond density and shape perspectives, the inherent asymmetry in certain topologies—where reversing the edges results in a topologically distinct configuration—has interested us in exploring the effects of reversed topologies. As shown in Figure 6, merely reversing the directions of specific topologies can lead to significant performance degradation. Typically, divergent topologies, characterized by having more child nodes than parent nodes, substantially outperform their convergent counterparts. Intuitively, artifact propagation diverges smoothly, enabling each agent to discuss artifacts from varied aspects. In contrast, aggregating multiple artifacts at a convergent node is more challenging, highlighting the complexity of integrating diverse aspects into a cohesive artifact. Therefore, to minimize potential degradation during artifact aggregation, it is recommended to employ topologies that maximize divergence while minimizing convergence.

The Ringelmann effect

The experiments ran in the 1880s.

> The **Ringelmann effect** is the tendency for individual members of a group to become increasingly less productive as the size of their group increases.[1] This effect, discovered by French agricultural engineer Maximilien Ringelmann (1861–1931), illustrates the inverse relationship that exists between the size of a group and the magnitude of group members' individual contribution to the completion of a task.
>
> While studying the relationship between process loss (i.e., reductions in performance effectiveness or efficiency) and group productivity, Ringelmann (1913) found that having group members work together on a task (e.g., pulling a rope) actually results in significantly less effort than when individual members are acting alone. Ringelmann discovered that as more and more people are added to a group, the group often becomes increasingly inefficient, ultimately violating the notion that group effort and team participation reliably leads to increased effort on behalf of the members.[1][2]

A follow-up study added blindfolded tug-of-war; similar findings.

Ringelmann effect, Wikipedia · Ringelmann rediscovered, JPSP 1986

Loss of coordination, the cause that transfers

When individuals come together in groups to perform a task, their performance depends on their individual resources (e.g., talents, skills, effort) and the various interpersonal processes at work within the group. Even if group members possess the abilities and expertise required to complete an assigned task, they may fail to coordinate their efforts in a productive way.[1] For example, hockey fans may feel that a particular team has the best chances of winning simply because the team is composed of all-star players. However, in reality, if the members of the team are not able to effectively synchronize their actions during game play, the team's overall performance will likely suffer. According to Steiner (1972), coordination problems between group members are a function of the demands of the tasks to be performed.[10] If a task is unitary (i.e., cannot be broken into subtasks for individual members), requires output maximization to be successful (i.e., a high rate of production quantity), and requires interdependence among members to yield a group product, the potential performance of a group relies on members' abilities to coordinate with one another.[1][10]

In people: motivation loss and coordination loss.

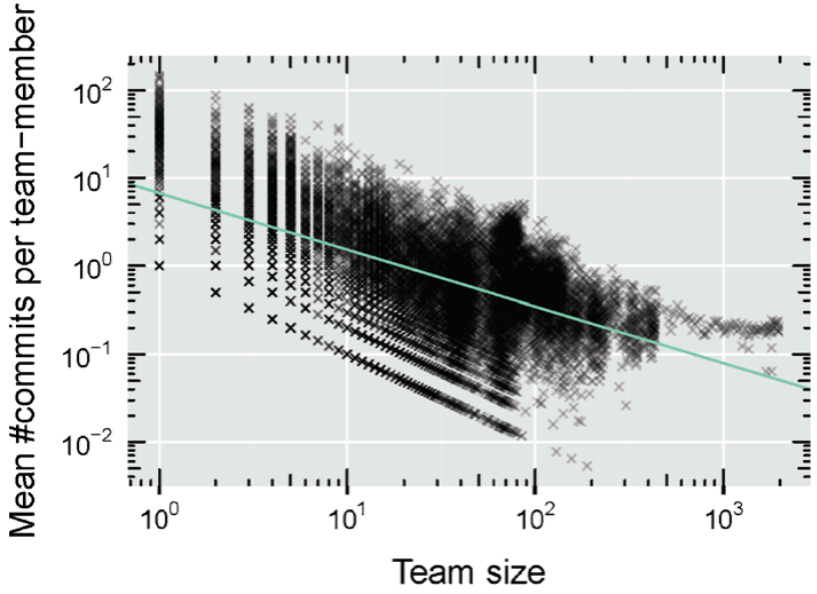
In agents: motivation loss may not be a driver (yet), but coordination remains a big factor.

Ringelmann effect, Wikipedia · Group process and productivity, Academic Press 1972

Software teams show the Ringelmann effect too

From Aristotle to Ringelmann: a large-scale analysis of team productivity and coordination in Open Source Software projects

Ingo Scholtes[1] · Pavlin Mavrodiev[1] · Frank Schweitzer[1]



Each × is one week of one project, pooled over 58 open-source projects.

Both axes are log scale.

Our findings confirm the negative relation between team size and productivity previously suggested by empirical software engineering research, thus providing quantitative evidence for the presence of a strong Ringelmann effect.

• • •

show that the magnitude of the decrease in productivity is likely to be related to the growth dynamics of co-editing networks which can be interpreted as a first-order approximation of coordination requirements.

Agents pay this coordination cost too. What must one more agent bring to be worth it?

From Aristotle to Ringelmann, Empirical Software Engineering 2016

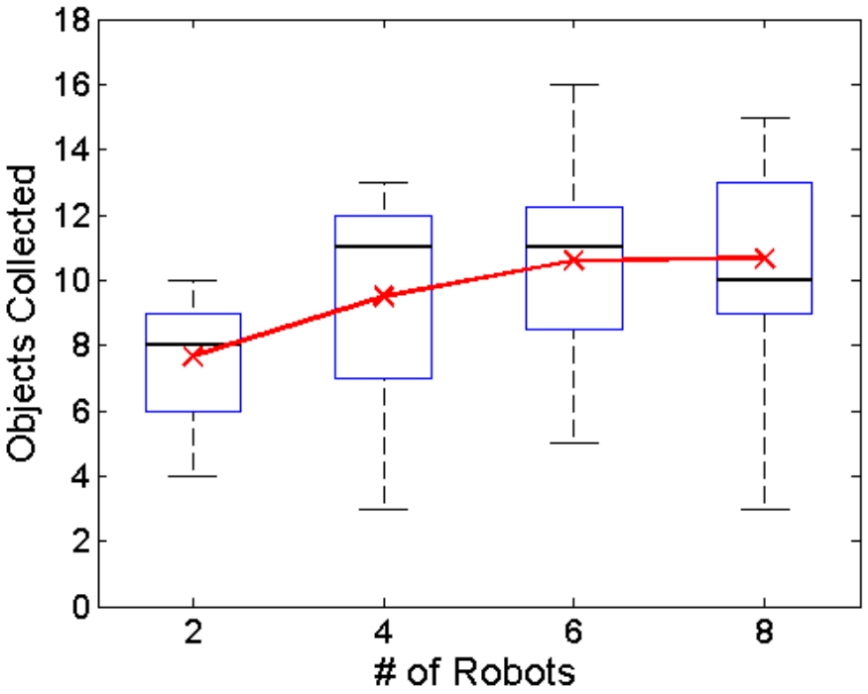
One person supervising robots: output plateaued at about six

IEEE TRANSACTIONS ON ROBOTICS – VOL. 23, NO. 5, OCTOBER 2007 1

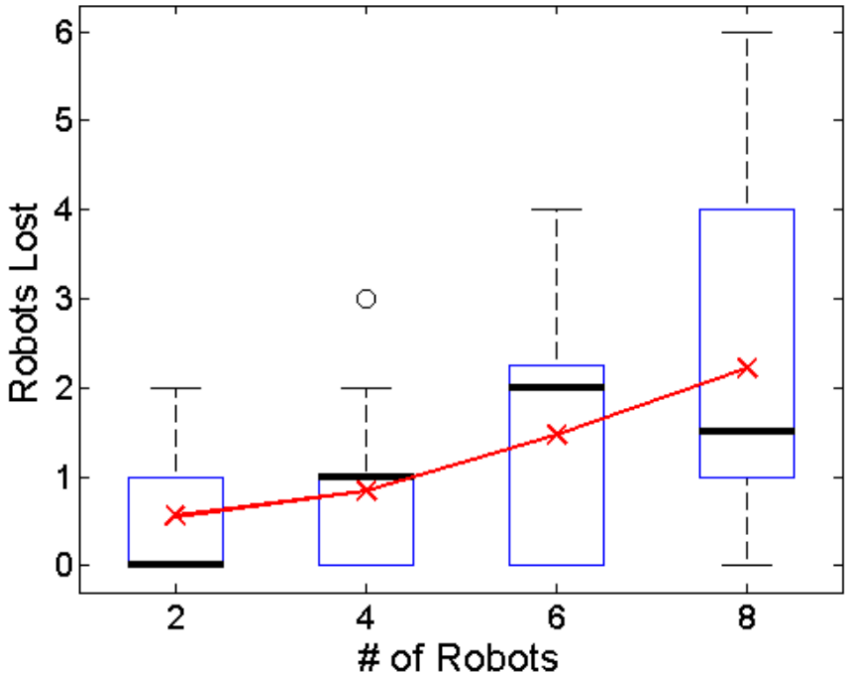
Identifying Predictive Metrics for Supervisory Control of Multiple Robots

Jacob W. Crandall and M. L. Cummings

One person directed 2 to 8 simulated robots through a maze for 8 minutes to collect objects. Robots still inside at the end were destroyed.



Objects collected: flat after six.



Robots lost: still climbing, a jump past four.

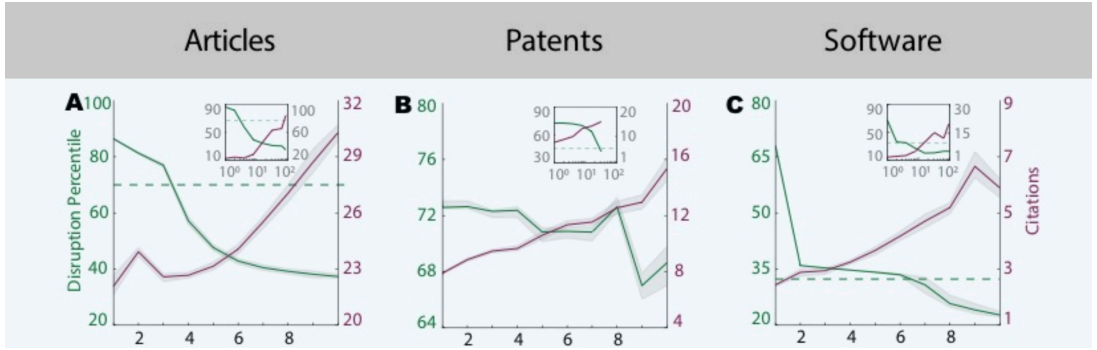
Large teams develop; small teams disrupt

Large teams develop and small teams disrupt science and technology

Lingfei Wu, Dashun Wang & James A. Evans

Team size also changes what kind of work you get. Evidence from human teams; no agent data.

x-axis: team size.



Here we analyse more than 65 million papers, patents and software products that span the period 1954–2014, and demonstrate that across this period smaller teams have tended to disrupt science and technology with new ideas and opportunities, whereas larger teams have tended to develop existing ones. Work from larger teams builds on more-recent and popular developments, and

• • •

individual, as people move between smaller and larger teams. These results demonstrate that both small and large teams are essential to a flourishing ecology of science and technology, and suggest that, to achieve this, science policies should aim to support a diversity of team sizes.

Large teams develop and small teams disrupt science and technology, Nature 2019

Outline

- A. Multi-agent in practice
- B. Dividing the work
- C. Coordinating the agents
- D. Running and checking the team

One agent first: Claude Code shrank its default team

> Agent teams add coordination overhead and use significantly more tokens than a single session. They work best when teammates can operate independently. For sequential tasks, same-file edits, or work with many dependencies, a single session or **subagents** are more effective.

Claude Code dynamic workflows: a smaller default

May 2026		July 2026
Tens to hundreds	→	**Fewer than 15**
advertised		default guideline

Adjustable; the release notes give no reason for the change.

Claude Code agent-team docs · 2.1.154, May 2026 · 2.1.219, Jul 2026

One agent first: debate added little beyond a prompt or a vote

Rethinking the Bounds of LLM Reasoning: Are Multi-Agent Discussions the Key?

Qineng Wang[1*] Zihao Wang[2*] Ying Su[2] Hanghang Tong[3] Yangqiu Song[2]

In this work, we reevaluate this claim through systematic experiments, where we propose a novel group discussion framework to enrich the set of discussion mechanisms. Interestingly, our results show that a single-agent LLM with strong prompts can achieve almost the same performance as the best existing discussion approach on a wide range of reasoning tasks and backbone LLMs. We observe that the multi-agent discussion performs better than a single agent only when there is no demonstration in the prompt. Further study reveals the common interaction mechanisms of LLMs during the discussion.

Debate or Vote: Which Yields Better Decisions in Multi-Agent Large Language Models?

Hyeong Kyu Choi Xiaojin Zhu Sharon Li*

Through extensive experiments across seven NLP benchmarks, we find that Majority Voting alone accounts for most of the performance gains typically attributed to MAD. To explain this, we propose a theoretical framework that models debate as a stochastic process. We prove that it induces a martingale over agents' belief trajectories, implying that debate alone does not improve expected correctness.

Rethinking the bounds of LLM reasoning, ACL 2024 · Debate or vote, NeurIPS 2025

One agent first: stronger single agents gain less from a team

Capable language models can outgrow the benefits of collaboration

Received: 20 December 2025

Accepted: 4 June 2026

Published online: 24 July 2026

Check for updates

Yubin Kim[1,2], Ken Gu[1], Chanwoo Park[2], Chunjong Park[3], Samuel Schmidgall[3], A. Ali Heydari[1], Yao Yan[1], Zhihan Zhang[1], Yuchen Zhuang[3], Yun Liu[1], Mark Malhotra[1], Paul Pu Liang[2], Hae Won Park[2], Yuzhe Yang[1], Xuhai Xu[1], Yilun Du[1], Shwetak Patel[1], Tim Althoff[1], Daniel McDuff[1] & Xin Liu[1]

Agents, language model-based systems that can reason, plan and act with tools to accomplish tasks, are widely deployed, yet it remains unclear when multi-agent coordination outperforms a strong single agent. Here we conduct a controlled experiment that holds task prompts, tools and compute budgets constant while varying only coordination structure and model capability. Across 260 configurations spanning six benchmarks, five architectures and three LLM families, we derive a predictive model using empirical coordination metrics. Across benchmarks, single-agent baseline performance emerges as the most robust predictor of whether coordination improves or decreases performance. In particular, we identify an empirical capability-saturation threshold beyond which additional agents are unlikely to improve performance. This threshold correctly predicts the effect of

Capable language models can outgrow the benefits of collaboration, Nature Machine Intelligence 2026

Persist needed context: Manus treats files as the agent's memory

A typical Manus task takes about 50 tool calls.

That's why we treat the **file system as the ultimate context** in Manus: unlimited in size, persistent by nature, and directly operable by the agent itself. The model learns to write to and read from files on demand—using the file system not just as storage, but as structured, externalized memory.

• • •

Manipulate Attention Through Recitation

If you've worked with Manus, you've probably noticed something curious: when handling complex tasks, it tends to create a **todo.md** file—and update it step-by-step as the task progresses, checking off completed items.

That's not just cute behavior—it's a deliberate mechanism to **manipulate attention**.

Context engineering for AI agents, Manus, Jul 2025

Persist needed context: every compiler session started empty

Put yourself in Claude's shoes

I had to constantly remind myself that I was writing this test harness for Claude and not for myself, which meant rethinking many of my assumptions about how tests should communicate results.

For example, each agent is dropped into a fresh container with no context and will spend significant time orienting itself, especially on large projects. Before we even reach the tests, to help Claude help itself, I included instructions to maintain extensive READMEs and progress files that should be updated frequently with the current status.

Building a C compiler with a team of parallel Claudes, Anthropic, Feb 2026

The verifier is the spec: compiler features kept breaking old ones

Lessons from programming with Claude agent teams

The scaffolding runs Claude in a loop, but that loop is only useful if Claude can tell how to make progress. Most of my effort went into designing the environment around Claude—the tests, the environment, the feedback—so that it could orient itself without me. These are the approaches I've found most helpful when orchestrating multiple Claude instances.

Write extremely high-quality tests

Claude will work autonomously to solve whatever problem I give it. So it's important that the task verifier is nearly perfect, otherwise Claude will solve the wrong problem. Improving the testing harness required finding high-quality compiler test suites, writing verifiers and build scripts for open-source software packages, and watching for mistakes Claude was making, then designing new tests as I identified those failure modes.

For example, near the end of the project, Claude started to frequently break existing functionality each time it implemented a new feature. To address this, I built a continuous integration pipeline and implemented stricter enforcement that allowed Claude to better test its work so that new commits can't break existing code.

Building a C compiler with a team of parallel Claudes, Anthropic, Feb 2026

The verifier is the spec: no globally locked merge queue

Feb 2026, Chris Lattner, who created LLVM and Swift, on the compiler:

> The most revealing parts of CCC are its mistakes. Several design choices suggest optimization toward passing tests rather than building general abstractions like a human would.

Oct 2026, Anthropic on its own source control:

> A proper merge queue, for example, doesn't make much sense at agent scale. What does CI even mean when hundreds or even thousands of changes can merge while tests are running? We haven't been able to effectively replicate the industry standard of a global repository lock on the merge queue, so instead, we've invested in post-merge and pre-deploy automated testing.

The compiler repository has had no commits since launch day; issue #1, "Hello world does not compile", is still open.

The Claude C Compiler, Chris Lattner, Modular, Feb 2026 · Redesigning source control, Anthropic, Oct 2026

The verifier is the spec: Anthropic separated judge and builder

Anthropic's harness for multi-hour app builds has three agents: planner, generator, evaluator.

A second issue, which we haven't previously addressed, is self-evaluation. When asked to evaluate work they've produced, agents tend to respond by confidently praising the work—even when, to a human observer, the quality is obviously mediocre. This problem is particularly pronounced for subjective tasks like design, where there is no binary check equivalent to a verifiable software test. Whether a layout feels polished or generic is a judgment call, and agents reliably skew positive when grading their own work.

However, even on tasks that do have verifiable outcomes, agents still sometimes exhibit poor judgment that impedes their performance while completing the task. Separating the agent doing the work from the agent judging it proves to be a strong lever to address this issue. The separation doesn't immediately eliminate that leniency on its own; the evaluator is still an LLM that is inclined to be generous towards LLM-generated outputs. But tuning a standalone evaluator to be skeptical turns out to be far more tractable than making a generator critical of its own work, and once that external feedback exists, the generator has something concrete to iterate against.

Harness design for long-running application development, Anthropic, Mar 2026

The verifier is the spec: Anthropic agreed on “done” before coding

Anthropic’s harness for multi-hour app builds has three agents: planner, generator, evaluator.

> Before each sprint, the generator and evaluator negotiated a sprint contract: agreeing on what "done" looked like for that chunk of work before any code was written. This existed because the product spec was intentionally high-level, and I wanted a step to bridge the gap between user stories and testable implementation. The generator proposed what it would build and how success would be verified, and the evaluator reviewed that proposal to make sure the generator was building the right thing. The two iterated until they agreed.
>
> Communication was handled via files: one agent would write a file, another agent would read it and respond either within that file or with a new file that the previous agent would read in turn. The generator then built against the agreed-upon contract before handing the work off to QA. This kept the work faithful to the spec without over-specifying implementation too early.

Harness design for long-running application development, Anthropic, Mar 2026

The verifier is the spec: Anthropic later dropped the sprints

Later in the same post, after Opus 4.6 came out:

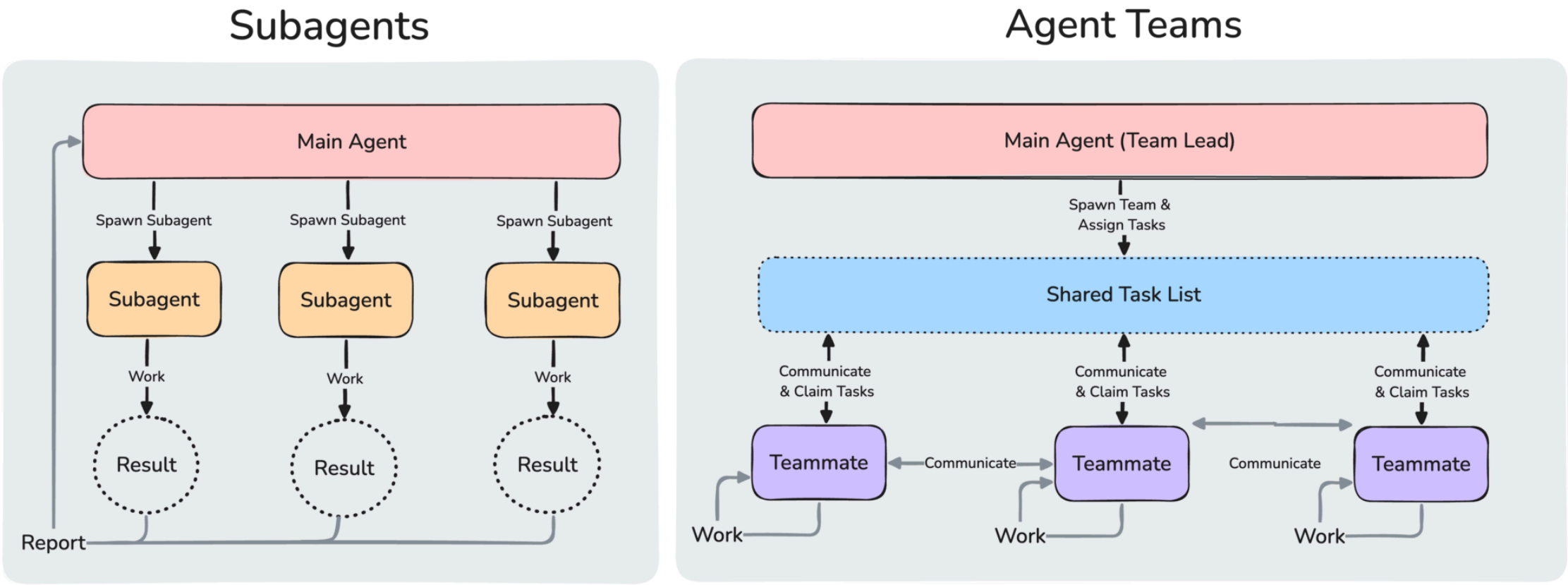
> I started by removing the sprint construct entirely. The sprint structure had helped to decompose work into chunks for the model to work coherently. Given the improvements in Opus 4.6, there was good reason to believe that the model could natively handle the job without this sort of decomposition.

The practical implication is that **the evaluator is not a fixed yes-or-no decision. It is worth the cost when the task sits beyond what the current model does reliably solo.**

Harness design for long-running application development, Anthropic, Mar 2026

Keep coordination simple: Claude Code is testing agent teams

> ⚠ Agent teams are experimental and disabled by default. Enable them by setting `CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1` in your **settings.json** or environment. Without that variable, no team is set up at session start, no team directories are written, and Claude does not spawn or propose teammates. Agent teams have **known limitations** around session resumption, task coordination, and shutdown



Orchestrate teams of Claude Code sessions, Claude Code docs, 2026

Keep coordination simple: mechanisms and ownership guidance

Dependency tracking: unresolved prerequisites block task claiming

The shared task list coordinates work across the team. The lead creates tasks and teammates work through them. Tasks have three states: pending, in progress, and completed. Tasks can also depend on other tasks: a pending task with unresolved dependencies cannot be claimed until those dependencies are completed.

File ownership is guidance, not an enforced write boundary

Two teammates editing the same file leads to overwrites. Break the work so each teammate owns a different set of files.

Devin: the manager assumed its child Devins knew what it knew

Teammates load project context automatically, including CLAUDE.md, MCP servers, and skills, but they don't inherit the lead's conversation history. See **Context and communication** for details. Include task-specific details in the spawn prompt:

A configured completion hook can block completion when its check fails

- `TaskCompleted`: runs when a task is being marked complete. Exit with code 2 to prevent completion and send feedback.

Orchestrate teams of Claude Code sessions, Claude Code docs, 2026

Review capacity caps the team: OpenAI's limit was human review

A product written entirely by Codex agents: about 1,500 merged pull requests in five months.

Humans may review pull requests, but aren't required to. Over time, we've pushed almost all review effort towards being handled agent-to-agent.

Increasing application legibility

As code throughput increased, our bottleneck became human QA capacity. Because the fixed constraint has been human time and attention, we've worked to add more capabilities to the agent by making things like the application UI, logs, and app metrics themselves directly legible to Codex.

The repository operates with minimal blocking merge gates. Pull requests are short-lived. Test flakes are often addressed with follow-up runs rather than blocking progress indefinitely. In a system where agent throughput far exceeds human attention, corrections are cheap, and waiting is expensive.

Review capacity caps the team: ten weeks later, context switching

Apr 2026, the same OpenAI team, after its harness post:

> And it worked, but then we ran into the next bottleneck: context switching.

Why:

> Each engineer would open a few Codex sessions, assign tasks, review the output, steer the agent, and repeat. In practice, most people could comfortably manage three to five sessions at a time before context switching became painful. Beyond that, productivity dropped.

An open-source spec for Codex orchestration: Symphony, OpenAI, Apr 2026

What the human owns

- Make the work divisible: pieces that can each be checked on their own.
- Write the brief. Objective, output, tools, boundaries; route discoveries yourself.
- Add judgment before writers. Seek different failure modes; another model is not automatically independent.
- Own acceptance: set the checks, risk limits, and escalation rules—not necessarily review every action.

Pick one task in your project. Would you hand it to one agent or several, and how would you check each piece?